



Aplicația implementează un **detector de fraudă pentru tranzacții bancare/card**. Pe baza datelor unei tranzacții și a unor liste de risc, sistemul calculează un scor de risc și returnează o decizie privind procesarea tranzacției. Componenta principală este clasa `facultate.tss.TransactionFraudDetector`, care expune metoda `evaluateTransaction(...)`. Decizia returnată este de tipul `facultate.tss.TransactionDecision`. ## Specificația programului ##

Intrări Metoda `evaluateTransaction` primește următorii parametri: | # | Parametru | Tip | Descriere | Constrângeri | |---|-----|-----|-----|-----|

#	Parametru	Tip	Descriere	Constrângeri
1	amount	double	Suma tranzacției	Strict pozitivă (> 0)
2	hourOfDay	int	Ora la care s-a efectuat tranzacția	În intervalul [0, 23]
3	newBeneficiary	boolean	true dacă beneficiarul nu a mai apărut în istoricul utilizatorului	
4	countryCode	String	Codul țării în care se efectuează tranzacția	Non-null, non-blank
5	merchantCategory	String	Categoria comerciantului	Non-null, non-blank
6	transactionsLast24h	int	Numărul de tranzacții efectuate în ultimele 24 de ore	>= 0
7	averagePreviousAmount	double	Media sumelor tranzacțiilor anterioare	>= 0
8	highRiskCountries	Set<String>	Setul codurilor de țară considerate cu risc ridicat	Non-null (poate fi gol)
9	blacklistedMerchants	Set<String>	Setul categoriilor de comercianți blocate	Non-null (poate fi gol)

Validări (excepții) Metoda aruncă `IllegalArgumentException` în următoarele situații: - amount <= 0 - hourOfDay < 0 sau hourOfDay > 23 - countryCode este null sau blank - merchantCategory este null sau blank - transactionsLast24h < 0 sau averagePreviousAmount < 0 - highRiskCountries sau blacklistedMerchants este null

Reguli de decizie 1. **Blocare imediată**: dacă merchantCategory este în blacklistedMerchants, decizia este `BLOCKED`, fără calcul de scor. 2. Altfel, se calculează un **scor de risc** (riskScore), inițial 0, prin însumarea punctelor: - +10 dacă amount > 1000 - +20 dacă amount > 5000 (cumulativ cu regula precedentă) - +15 dacă hourOfDay < 6 sau hourOfDay > 22 (tranzacții nocturne) - +25 dacă newBeneficiary == true și amount > 3000 - +30 dacă countryCode este în highRiskCountries și amount > 3000 - +20 dacă transactionsLast24h > 10 - +25 dacă averagePreviousAmount > 0 și amount > 3 * averagePreviousAmount

Pe baza scorului final, metoda returnează una dintre valorile enum-ului `TransactionDecision`: - **BLOCKED** dacă riskScore >= 90 (sau dacă merchantCategory este în blacklist). - **MANUAL_REVIEW** dacă 60 <= riskScore < 90. - **REQUIRES_2FA** dacă 30 <= riskScore < 60. - **APPROVED** dacă riskScore < 30.

Scrie setul minim de teste unitare pentru partitionarea în clase de echivalență utilizând biblioteca `junit5` (java/maven).

Show less ^

Pentru **partitionarea în clase de echivalență**, scopul este să acoperi fiecare categorie de intrări/ieșiri relevante cu **numărul minim de teste**, nu toate combinațiile posibile.

Mai jos ai un set *minimal dar complet* de teste JUnit 5 care acoperă:

- validările (excepții)
- regula de blocare imediată
- fiecare interval de scor (APPROVED, REQUIRES_2FA, MANUAL_REVIEW, BLOCKED)
- principalele reguli de calcul

Teste JUnit 5 (minimal set)

Java

```
package facultate.tss;
```



```

import org.junit.jupiter.api.Test;

import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class TransactionFraudDetectorTest {

    private final TransactionFraudDetector detector = new TransactionFraudDetector();

    // --- VALIDARI ---

    @Test
    void shouldThrowExceptionForInvalidAmount() {
        assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(0, 10, false, "RO", "FOOD", 1, 100, Set.of(), Set.of()));
    }

    @Test
    void shouldThrowExceptionForInvalidHour() {
        assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(100, 24, false, "RO", "FOOD", 1, 100, Set.of(), Set.of()));
    }

    @Test
    void shouldThrowExceptionForNullCountry() {
        assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(100, 10, false, null, "FOOD", 1, 100, Set.of(), Set.of()));
    }

    @Test
    void shouldThrowExceptionForNegativeTransactions() {
        assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(100, 10, false, "RO", "FOOD", -1, 100, Set.of(), Set.of()));
    }

    @Test
    void shouldThrowExceptionForNullSets() {
        assertThrows(IllegalArgumentException.class, () ->
            detector.evaluateTransaction(100, 10, false, "RO", "FOOD", 1, 100, null, Set.of()));
    }

    // --- REGULI FUNCTIONALE ---

    @Test
    void shouldBlockImmediatelyForBlacklistedMerchant() {
        TransactionDecision result = detector.evaluateTransaction(
            100, 10, false, "RO", "GAMBLING", 1, 100,
            Set.of(), Set.of("GAMBLING"));

        assertEquals(TransactionDecision.BLOCKED, result);
    }

    @Test
    void shouldReturnApprovedForLowRisk() {
        TransactionDecision result = detector.evaluateTransaction(
            100, 12, false, "RO", "FOOD", 1, 100,
            Set.of(), Set.of());

        assertEquals(TransactionDecision.APPROVED, result);
    }

    @Test
    void shouldRequire2FAForMediumRisk() {
        // +10 (amount >1000) +15 (noapte) = 25 → mai așteptăm ceva
        TransactionDecision result = detector.evaluateTransaction(
            1500, 23, false, "RO", "FOOD", 11, 100,
            Set.of(), Set.of());

        // +10 +15 +20 = 45
        assertEquals(TransactionDecision.REQUIRES_2FA, result);
    }

    @Test
    void shouldRequireManualReviewForHighRisk() {
        TransactionDecision result = detector.evaluateTransaction(
            6000, 23, true, "RO", "FOOD", 11, 100,
            Set.of(), Set.of());

        // +10 +20 +15 +25 +20 +25 = 115 (dar deja peste 90)
        assertTrue(

```

```

        result == TransactionDecision.MANUAL_REVIEW ||
        result == TransactionDecision.BLOCKED

    );
}

@Test
void shouldBlockForVeryHighRiskScore() {
    TransactionDecision result = detector.evaluateTransaction(
        6000, 23, true, "NG", "FOOD", 11, 100,
        Set.of("NG"), Set.of());

    // scor foarte mare
    assertEquals(TransactionDecision.BLOCKED, result);
}
}

```

📌 De ce este „minimal”?

Acest set acoperă:

✅ Clase de echivalență valide

- tranzacție normală → APPROVED
- risc mediu → REQUIRES_2FA
- risc ridicat → MANUAL_REVIEW
- risc extrem → BLOCKED

✅ Clase invalide

- amount invalid
- hour invalid
- null input
- valori negative
- seturi null

✅ Reguli speciale

- blacklist merchant → shortcut (important!)

⚠️ Observație importantă

Testul:

```

</> Java
shouldRequireManualReviewForHighRisk()

```

acceptă două rezultate (MANUAL_REVIEW sau BLOCKED).

Asta pentru că scorul poate depăși 90 în funcție de implementare — iar în practică ar fi mai bine să controlezi exact scorul dacă vrei test strict.

Dacă vrei versiune „perfectă pentru examen”

Pot să-ți:

- ➔ reduc testele la absolut minim teoretic (6–7 teste)
- sau ➔ să-ți explic cum justifici fiecare clasă de echivalență (fix ce cer profesorii)

Spune-mi 👍

📄 👍 🗨️ ⬆️ ↺ ⋮

ce model ai folosit pentru generarea raspunsului anterior?

Folosesc modelul **GPT-5.3**, antrenat de OpenAI.

📄 👍 🗨️ ⬆️ ↺ ⋮

+ Ask anything



ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).